IFN647 Week 9 Review Questions

Question 1 (<u>Multinomial Document Representation</u>)
Table 1 shows an illustration of how documents are represented in the multinomial event space. In this table, there are 10 documents (each with a unique document id), two class labels (spam and not_spam), and a vocabulary that consists of the terms "cheap", "buy", "banking", "dinner", and "the".

Table 1. A training set (multinomial event space)

document *id*	cheap	buy	banking	dinner	the	*class*
1	0	0	0	0	2	not spam
2	3	0	1	0	1	spam
3	0	0	0	0	1	not spam
4	2	0	3	0	2	spam
5	5	2	0	0	1	spam
6	0	0	1	0	1	not spam
7	0	1	1	0	1	not spam
8	0	0	0	0	1	not spam
9	0	0	0	0	1	not spam
10	1	1	0	1	2	not spam

Based on Table 1, we can represent this training set for Naïve Bayes classification as follows:

$D = \{d_1, d_2, \ldots, d_{10}\}$, $\boldsymbol{C} = \{c_1, c_2\}$ = {'*spam*', '*not spam*'},

$V = \{w_1, w_2, \ldots, w_5\}$ = {'cheep', 'buy', 'banking', 'dinner', 'the'}

where D is the training set, $\boldsymbol{C}$ is the set of class labels, and V is the set of terms (vocabulary).

We can use the following equation to calculate the condition probability $P(c|d)$

$$P(c|d) = \log\big(P(c)\big) + \sum_{w_i \in d} \log\left(P(w_i|c)\right)$$

(1) Based on Naïve Bayes Algorithm, calculate $P(c)$ for all $c \in \boldsymbol{C}$.
(2) Based on Naïve Bayes Algorithm, calculate $P(w_i|c_j)$ for all $w_i \in V$ and all $c_j \in \boldsymbol{C}$.

Question 2 (<u>Naive Bayes classification</u>)
Let $\{c_0, c_1\}$ be a set of classes and D be a training set, where each element in D is a pair (d_j, c_j), d_j is a document and c_j is its class. For example, Table 1 shows a training set D which contains 10 documents and their multinomial document representations.

Assume this training set is represented as two Python lists as follows:

D = [[0,0,0,0,2],[3,0,1,0,1],[0,0,0,0,1],[2,0,3,0,2],[5,2,0,0,1],[0,0,1,0,1],[0,1,1,0,1],[0,0,0,0,1],[0,0,0,0,1],[1,1,0,1,2]]

C = [0,1,0,1,1,0,0,0,0,0]

where a document is represented as a list of terms' frequency counts in the document (please note we assume the vocabulary set V= ['cheep', 'buy', 'banking', 'dinner', 'the'] is numbered from 0 to n-1), D is represented as a list of documents, C is represented as a list of class labels of the corresponding documents in D, and c_1 (spam) and c_0 (not spam) are labelled as 1 and 0 in C.

The training procedure of Naive Bayes classification is to calculates prior probability $P(c)$ and conditional probabilities $P(w|c)$ for all terms w in V and all classes c in $\{c_0, c_1\}$, where $P(c) = N_c / |D|$, $P(w|c) = (tf_{w,c} + 1)/(|c| + |V|)$, N_c is the number of training documents with class label c, $tf_{w,c}$ is the number of times that term w occurs in class c in the training set, and $|c|$ is the total number of times of terms that occur in training documents with class label c.

It then uses Bayes' Rule to compute the probability of class c_i occurring for the given document d, $P(c_i|d)$ (i =0 or 1) for all document d, and assigns a class label c_i to document d if c_i is the highest probability of being computed given the document.

(1) Define a python function *TRAIN_MULTINOMIAL_NB*(C, D, V) to calculate prior probability $P(c)$ and conditional probabilities $P(w_j|c_i)$. Please note the function parameters are different to the Naive Bayes Algorithm in the lecture notes.

(2) Define a python function *APPLY_MULTINOMIAL_NB*(*V, prior, condprob, d*) to assign a label (1 or 0) to document d, where d is represented as a list of words.

Question 3 (*Rocchio* Classification)

Let C be a set of classes and D be a training set, where each element in D is a pair (d_j, c_j), d_j is a document and c_j is its class. The training procedure of Rocchio classification for the input training set D can be found in the lecture notes. It uses centroids to define the boundaries between classes. The centroid of each class is computed as the vector average of its members.

For the given topic "R101", we have obtained the following training set which includes seven documents, ten selected features (terms) (VM, US, Spy, Sale, Man, GM, Espionag, Econom, Chief, and Bill) and the corresponding term weights, where Class = 1 means relevant, and Class = 0 means non-relevant.

Doc	VM	US	Spy	Sale	Man	GM	Espionag	Econom	Chief	Bill	***Class***
'39496'	0.17	0.01	0.20	0.00	0.02	0.20	0.12	0.11	0.00	0.00	1
'46547'	0.10	0.21	0.10	0.00	0.00	0.00	0.22	0.20	0.00	0.10	1
'46974'	0.00	0.23	0.10	0.20	0.05	0.10	0.10	0.10	0.01	0.00	1
'62325'	0.17	0.01	0.20	0.00	0.02	0.20	0.12	0.11	0.00	0.00	1
'6146'	0.10	0.00	0.00	0.30	0.10	0.20	0.00	0.12	0.10	0.00	0
'18586'	0.00	0.30	0.00	0.30	0.20	0.00	0.00	0.20	0.15	0.20	0
'22170'	0.20	0.00	0.00	0.15	0.20	0.25	0.00	0.00	0.00	0.10	0

Assume the above training set is represented as two Python dictionaries as follows:

Question 3 continued overleaf

Question 3 continued

document_vectors = {
'39496': {'VM': 0.17, 'US': 0.01, 'Spy': 0.20, 'Sale': 0.00, 'Man': 0.02, 'GM': 0.20, 'Espionag': 0.12, 'Econom': 0.11, 'Chief': 0.00, 'Bill': 0.00},
'46547': {'VM': 0.10, 'US': 0.21, 'Spy': 0.10, 'Sale': 0.00, 'Man': 0.00, 'GM': 0.00, 'Espionag': 0.22, 'Econom': 0.20, 'Chief': 0.00, 'Bill': 0.10},
'46974': {'VM': 0.00, 'US': 0.23, 'Spy': 0.10, 'Sale': 0.20, 'Man': 0.05, 'GM': 0.10, 'Espionag': 0.10, 'Econom': 0.10, 'Chief': 0.01, 'Bill': 0.00},
'62325': {'VM': 0.17, 'US': 0.01, 'Spy': 0.20, 'Sale': 0.00, 'Man': 0.02, 'GM': 0.20, 'Espionag': 0.12, 'Econom': 0.11, 'Chief': 0.00, 'Bill': 0.00},
'6146': {'VM': 0.10, 'US': 0.00, 'Spy': 0.00, 'Sale': 0.30, 'Man': 0.10, 'GM': 0.20, 'Espionag': 0.00, 'Econom': 0.12, 'Chief': 0.10, 'Bill': 0.00},
'18586': {'VM': 0.00, 'US': 0.30, 'Spy': 0.00, 'Sale': 0.30, 'Man': 0.20, 'GM': 0.00, 'Espionag': 0.00, 'Econom': 0.20, 'Chief': 0.15, 'Bill': 0.20},
'22170': {'VM': 0.20, 'US': 0.00, 'Spy': 0.00, 'Sale': 0.15, 'Man': 0.20, 'GM': 0.25, 'Espionag': 0.00, 'Econom': 0.00, 'Chief': 0.00, 'Bill': 0.10} }

relevance_judgements = {'R101': {'0': ['6146', '18586', '22170'], '1': ['39496', '46547', '46974', '62325']}}

Let *given_topic* = 'R101', design a Python function *train_Rocchio*(*document_vectors*, *relevance_judgements*, *given_topic*) to calculate the centroid of the relevant class (named as 'R101') and the centroid of non-relevant class (named as 'nR101'), respectively. For example, use the above table, we have 'nR101' = {'VM': 0.100, 'US': 0.100, 'Spy': 0.000, 'Sale': 0.250, 'Man': 0.167, 'GM': 0.150, 'Espionag': 0.000, 'Econom': 0.107, 'Chief': 0.083, 'Bill': 0.100}.

Question 4 Which of the following is false for **sentiment analysis**? and justify your answer.

(1) Sentiment analysis (opinion mining) discovers users' opinions about products or services in on-line reviews or feedback or observes trends in public mood to analysis of clinical records.
(2) The orientation is the opinion provided about the entity and/or the aspect that was provided by the opinion holder at a specific time.
(3) The goal of emotion classification is to separate subjective from objective information, a binary classification task.
(4) Aspects are features, components or functions of the entity. They can be nouns and/or noun phrases
(5) Polarity classification is to group the expressed opinion in a document, a sentence or an entity feature/aspect in positive, negative or neutral regions.

Question 5 (**Sentiment Analysis**)

Consider the following sentence:

"The mobile phone has excellent build quality and performs efficiently when running games. Its camera quality is outstanding; however, the battery life is disappointing."

a) Using NLTK, define a get_aspect() function that identifies all *aspects* in the sentence by extracting tokens whose part-of-speech (POS) tags begin with **"**NN**"** (i.e., nouns and noun variants such as NN, NNS, NNP, NNPS). You may use the word_tokenize and pos_tag functions.
b) Design a prompt based on the Aspect-Based Sentiment Analysis (ABSA) model to extract all aspects mentioned in the sentence.
c) Compare and discuss the results obtained in a) and b), highlighting similarities, differences, and limitations of each approach.